

Guide to the `lsmethod` project structure

May 15, 2026

1 Purpose of the project

The purpose of `lsmethod` is to make the local subtraction calculations reproducible. The project is meant to connect three things:

1. the LaTeX derivation, with stable equation labels,
2. symbolic algebra checks, mainly with `Symbolica`,
3. numerical checks, mainly with `mpmath` and `pytest`.

The current target object is the two-loop differential collinear kernel

$$\mathcal{I}(x, y) \tag{1}$$

with support

$$0 \leq x \leq 1, \quad 0 \leq y \leq 1 - x. \tag{2}$$

The code is not meant to replace the derivation. It is meant to test whether the different analytic representations of the same object agree.

The relevant analytic logic is similar to the one-loop structure in the reference paper: a universal collinear kernel is integrated against a remaining hard or subgraph factor. In the paper this appears as

$$I_G = \int dx I_c(x, m, M) G(x), \tag{3}$$

and in the present two-loop setup the corresponding object is schematically

$$I_G^{(2)} = \int_0^1 dx \int_0^{1-x} dy \mathcal{I}(x, y) G(x, y). \tag{4}$$

The paper also emphasizes that integrated counterterms are simpler than the original diagram and can be used to extract singular dependence. This is the reason why the code starts by comparing reduced representations, rather than attempting a direct two-loop Minkowski integration.

2 High-level folder structure

The scaffold is organized as follows.

```
lsmethod-scaffold/  
  README.md  
  environment.yml  
  pyproject.toml  
  .gitignore  
  .vscode/settings.json  
  
src/lsmethod/
```

```

__init__.py
kinematics.py
kernels.py
numeric.py
equations.py
symbolica_backend.py
expansions.py

tests/
  test_closed_vs_w.py
  test_closed_vs_u.py
  test_support_region.py
  test_equation_registry.py

scripts/
  check_closed_vs_w.py
  check_closed_vs_u.py
  scan_random_points.py

docs/
  equation_map.yml

derivations/snippets/
  double_collinear_labels.tex

symbolica/
  check_import.py

cluster/
  submit_scan.sh

```

The central rule is:

Code that is reused goes into `src/lsmethod`. Code that is run manually goes into `scripts`. Automatic checks go into `tests`. LaTeX labels are mapped in `equations.py` and `docs/equation_map.yml`.

3 The two setup files

3.1 `environment.yml`

This file defines the conda or mamba environment. It specifies which Python version and which external packages should be installed.

```

name: lsmethod
channels:
  - conda-forge
dependencies:
  - python=3.12
  - numpy
  - scipy
  - mpmath
  - pandas
  - matplotlib
  - jupyterlab
  - ipykernel
  - pytest
  - pip

```

```
- pip:
  - symbolica
```

Use it with

```
mamba env create -f environment.yml
mamba activate lsmethod
```

This is the reproducible numerical laboratory. If the code later runs on Euler, this file tells you which packages are needed.

3.2 pyproject.toml

This file defines the Python project itself. It says that the package is called `lsmethod`, that the code lives under `src`, and that `pytest` should look in the `tests` folder.

```
[project]
name = "lsmethod"
version = "0.1.0"

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["src"]
```

The command

```
python -m pip install -e .
```

installs the local package in editable mode. Editable mode means that changes in `src/lsmethod` are visible immediately, without reinstalling.

The Python packaging guide describes `pyproject.toml` as the central configuration file for packaging tools and other project tools. Pytest can also read configuration from project-level files in the root of the repository.

4 The package `src/lsmethod`

4.1 `kinematics.py`

This file stores one numerical phase-space point. The main object is

```
@dataclass(frozen=True)
class Kinematics:
    x: mp.mpf
    y: mp.mpf
    p2: mp.mpf
    ml: mp.mpf
    Ml: mp.mpf
    mk: mp.mpf
    Mk: mp.mpf
```

A `dataclass` is a compact way of defining a class whose main purpose is to store data. Without `dataclass`, one would have to write the `__init__` method manually. With `frozen=True`, the object cannot be changed after creation. This is useful because a kinematic point should be fixed once it is passed to a kernel function.

The fields mean

$$x, \quad y, \quad p^2, \quad m_l, \quad M_l, \quad m_k, \quad M_k. \quad (5)$$

The derived properties are

$$X = 1 - x, \quad (6)$$

$$\lambda = \frac{y}{1 - x}, \quad (7)$$

$$\kappa = \lambda(1 - \lambda). \quad (8)$$

The mass combinations are

$$\mu_l^2(x) = (1 - x)m_l^2 + xM_l^2, \quad (9)$$

$$\mu_l^2(u) = um_l^2 + (1 - u)M_l^2, \quad (10)$$

$$\mu_k^2(\lambda) = (1 - \lambda)m_k^2 + \lambda M_k^2. \quad (11)$$

The reason for using a `Kinematics` object is that all formulas depend on the same variables. Instead of writing

```
closed_form(x, y, p2, ml, Ml, mk, Mk, eps)
```

one writes

```
closed_form(kin, eps)
```

where `kin` is the fixed point.

4.2 mpmath and mp

The line

```
import mpmath as mp
```

imports `mpmath` under the shorter name `mp`. `Mpmath` is used because it supports arbitrary-precision real and complex arithmetic. This matters for endpoint-singular integrals, Gamma functions, and small values of ϵ .

A typical setup is

```
mp.mp.dps = 50
x = mp.mpf("0.30")
```

Here `dps` means decimal places. It is better to write `mp.mpf("0.30")` instead of `mp.mpf(0.30)`, because the string version avoids first creating a standard double-precision Python float.

4.3 kernels.py

This is the central physics file. It contains the numerical implementations of the equations in the LaTeX file.

The main functions are:

Function	Meaning	LaTeX label
<code>delta0(kin)</code>	first effective scale	<code>eq:Delta0-final</code>
<code>delta1(kin)</code>	second effective scale	<code>eq:Delta1-final</code>
<code>closed_form(kin, eps)</code>	compact kernel	<code>eq:closed-form-kernel</code>
<code>delta_w(kin)</code>	slope in $\Delta_x(w)$	<code>eq:method1-Delta0-Deltaw-def</code>
<code>delta_x_w(kin,w)</code>	$\Delta_x(w)$	<code>eq:method1-Delta-x-def</code>
<code>method1_w_integral</code>	Method 1 representation	<code>eq:method1-w-integral-before-euler</code>
<code>C_u(kin,u)</code>	Method 2 scale	<code>eq:C-u-def</code>
<code>method2_u_integral</code>	Method 2 representation	<code>eq:method1-u-representation</code>

The function `closed_form` implements

$$\mathcal{I}_{\text{closed}}(x, y; \epsilon) = \frac{\Gamma(\epsilon)^2}{1-x} \Theta_{xy} [\Delta_0 - i0]^{-\epsilon} [\Delta_1 - i0]^{-\epsilon}. \quad (12)$$

The functions `method1_w_integral` and `method2_u_integral` implement two reduced parameter integrals that should agree with the closed form.

4.4 `numeric.py`

This file contains small utilities that are not specific to the physics formula.

```
def relative_error(value, reference):
    if abs(reference) == 0:
        return abs(value - reference)
    return abs((value - reference) / reference)
```

This compares two numerical values. It uses an absolute error if the reference is exactly zero.

```
def quad_split(function, a, b, parts=4):
    points = [a + (b - a) * i / parts for i in range(parts + 1)]
    return mp.quad(function, points)
```

This splits an integral interval into subintervals. This is useful because integrands like

$$u^{-1+\epsilon}(X-u)^{-1+\epsilon} \quad (13)$$

are singular at endpoints for small positive ϵ .

4.5 `equations.py` and `docs/equation_map.yml`

These files connect code names to LaTeX labels. Example:

```
EQUATIONS = {
    "delta0": "eq:Delta0-final",
    "delta1": "eq:Delta1-final",
    "closed_form": "eq:closed-form-kernel",
}
```

The reason for labels is simple. Equation numbers change when the LaTeX document changes. Labels should remain stable. Therefore the code should never say `Eq. (25)`. It should say `eq:Delta1-final`.

4.6 `symbolica_backend.py`

At the moment this file is only a placeholder:

```
def check_symbolica_import():
    import symbolica
    return symbolica
```

The reason is that the numerical layer is already useful, while the symbolic layer should be built carefully. The intended future role of this file is to hold Symbolica expressions for Δ_0 , Δ_1 , Δ_w , C_u , and their algebraic identities.

5 What does pytest do?

5.1 The idea

`pytest` runs automatic checks. It looks for files whose names start with `test_` and then runs functions whose names also start with `test_`. In this project, the configuration is in `pyproject.toml`:

```
[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["src"]
```

This tells `pytest`:

- look for tests in `tests`,
- when importing code, also look in `src`.

Run all tests with

```
pytest
```

Run one test file with

```
pytest tests/test_closed_vs_w.py
```

Run with verbose output using

```
pytest -v
```

5.2 What the current tests do

The current tests are smoke tests and structural tests. They check that the basic functions are importable and evaluable.

tests/test_support_region.py This checks that the closed form returns zero outside the support:

$$y > 1 - x \quad \Rightarrow \quad \mathcal{I}(x, y) = 0. \quad (14)$$

tests/test_equation_registry.py This checks that important code names have LaTeX labels in `EQUATIONS`. It does not check the mathematics. It checks that the code-to-LaTeX map has not disappeared.

tests/test_closed_vs_w.py This currently checks that the closed form and the Method 1 integral can both be evaluated at a Euclidean test point.

tests/test_closed_vs_u.py This currently checks that the closed form and the Method 2 integral can both be evaluated at a Euclidean test point.

A stronger future version should check equality within a tolerance:

```
from lsmethod.numeric import relative_error

assert relative_error(numeric, exact) < mp.mpf("1e-25")
```

This stricter test should be enabled once the formulas in the code and in the LaTeX file are finalized.

6 How to do simple checks

6.1 Check that the package imports

From the project root:

```
python -c "import lsmethod; print(lsmethod)"
```

6.2 Check the Symbolica installation

```
python symbolica/check_import.py
```

6.3 Evaluate one kinematic point

```
python scripts/check_closed_vs_w.py
python scripts/check_closed_vs_u.py
```

These scripts print the closed form, the corresponding parameter-integral representation, and the relative error.

6.4 Run a small scan

```
python scripts/scan_random_points.py --npoints 20 --eps 0.08
```

This writes a CSV file to

```
results/numeric_checks/scan.csv
```

The scan is useful for finding regions where numerical integration becomes unstable.

7 How to get symbolic output

There are two different goals:

1. print the formula associated with a code function,
2. manipulate the formula symbolically and check identities.

7.1 Simple formula output

The simplest robust method is to store LaTeX strings for important equations. For example, one can create a file `src/lsmethod/formulas.py`:

```
FORMULAS = {
    "delta0": r"\Delta_0 = \mu_1^2(x)-x(1-x)p^2",
    "delta1": r"\Delta_1 = \mu_k^2(\lambda)-\lambda(1-\lambda)M_1^2-\lambda(1-\lambda)x)p^2",
}

def latex_formula(name):
    return FORMULAS[name]
```

Then use

```
from lsmethod.formulas import latex_formula
print(latex_formula("delta1"))
```

This gives clean output for notes, notebooks, or Overleaf snippets. It does not do algebra.

7.2 Symbolica expressions

Symbolica should be used for actual symbolic manipulation. According to the Symbolica Python documentation, expressions can be built with `Expression.var`, parsed from strings with `Expression.parse`, and printed as LaTeX using `pretty_str(latex=True)`.

A future version of `symbolica_backend.py` could look like:

```
from symbolica import Expression

def variables():
    return Expression.vars("x", "y", "p2", "m1", "M1", "mk", "Mk")

def delta0_expr():
    x, y, p2, m1, M1, mk, Mk = variables()
    mu_l_x = (1 - x) * m1**2 + x * M1**2
    return mu_l_x - x * (1 - x) * p2

def print_delta0_latex():
    print(delta0_expr().pretty_str(latex=True))
```

This has two advantages:

- the symbolic expression is independent of a numerical point,
- the same expression can be printed, simplified, substituted, or compared.

The safe project rule is:

Numerical functions in `kernels.py` should have matching symbolic functions in `symbolica_backend.py` whenever the expression is central to the derivation.

8 The actual numerical validation idea

8.1 Internal consistency is not the final validation

The equality

$$\mathcal{I}_{\text{closed}} = \mathcal{I}_w = \mathcal{I}_u \quad (15)$$

should not be presented as the main physics cross-check. It is an internal consistency and implementation check only.

It verifies that three expressions already derived in the same calculation have been implemented consistently:

- $\mathcal{I}_w$ is the reduced Method 1 parameter representation,
- $\mathcal{I}_u$ is the reduced Method 2 parameter representation,
- $\mathcal{I}_{\text{closed}}$ is the closed form obtained after the final analytic reduction.

This check is still useful. It can catch typos in prefactors, support factors, powers, and definitions such as Δ_0 , Δ_1 , Δ_w , or C_u . But it does not prove that the final result is correct as a representation of the original two-loop integral. Since Method 1 and Method 2 have already been analytically matched, comparing their code implementations is a regression test, not an independent validation.

8.2 What is the actual goal?

The real goal is to validate the closed form against something outside the final derivation. There are three increasingly strong levels:

1. Compare against less reduced representations that are closer to the original loop integral.
2. Compare integrated quantities against known results or known limiting cases.
3. If possible, compare against a direct numerical evaluation of the original differential integral with a controlled contour or Euclidean formulation.

The project should therefore distinguish two types of checks:

Type	Example	Status
Internal code check	$\mathcal{I}_{\text{closed}} = \mathcal{I}_w = \mathcal{I}_u$	useful, but not independent
External validation	compare to original integral or literature	actual physics check

8.3 What can be compared to the original integral?

The original object is the differential two-loop integral with two delta constraints. Schematically,

$$\mathcal{I}(x, y) = \int d^d l d^d k \delta(x - l \cdot \eta) \delta(y - k \cdot \eta) \frac{1}{D_l D_{p-l} D_k D_{p-l-k}}. \quad (16)$$

A direct numerical integration of this expression on real Minkowski loop momenta is not a simple multidimensional real integral. The propagators carry $i0$ prescriptions, and a naive real-axis integration will run into pole and contour issues. This is the same general reason why the paper separates singular counterterm integrals from finite remainders and discusses numerical evaluation only after the singular regions have been removed or analytically isolated.

However, one can still move systematically closer to the original integral. A sensible chain of increasingly independent checks is:

$$\text{original differential loop integral} \rightarrow \text{residue representation before transverse integration} \rightarrow \text{transverse integral} \quad (17)$$

The current code only starts near the end of this chain. To make the check more meaningful, the project should add earlier representations.

8.4 Recommended validation hierarchy

Level 0: smoke tests. Check that all functions run and return finite complex numbers at safe Euclidean test points. This is not a mathematical validation. It only checks that the code is wired correctly.

Level 1: internal regression tests. Check

$$\mathcal{I}_{\text{closed}} = \mathcal{I}_w = \mathcal{I}_u. \quad (18)$$

This is useful after every code or algebra edit, but it should be described as an internal consistency test.

Level 2: symbolic identity checks. Use Symbolica to verify algebraic relations that are used in the derivation, for example

$$\Delta_x(w) = \Delta_0 + w\Delta_w, \quad (19)$$

$$\Delta_1 = -[(1 + \kappa)\Delta_0 + \Delta_w], \quad (20)$$

where the second identity depends on the sign conventions used in the file. These tests are more useful than purely numerical equality between already matched final representations because they test specific algebraic steps.

Level 3: numerical check against an earlier representation. Implement a representation before the final parameter integrations are done. For example, keep the expression after the light-cone residue step and do the remaining transverse or Feynman-parameter integrations numerically. This is closer to the original integral than the final closed form.

A useful target is a representation where the β_k residue has been taken, but the transverse integrations have not yet been done analytically. Numerically integrating that object and comparing it to the closed form would be a stronger check than comparing $\mathcal{I}_w$ and $\mathcal{I}_u$.

Level 4: integrated tests with simple hard functions. The physically useful object is not only $\mathcal{I}(x, y)$, but the convolution

$$I_G^{(2)} = \int_0^1 dx \int_0^{1-x} dy \mathcal{I}(x, y) G(x, y). \quad (21)$$

For test functions such as

$$G(x, y) = 1, \quad (22)$$

$$G(x, y) = \frac{1}{1 + x + y}, \quad (23)$$

one can integrate the closed form over (x, y) and compare to a numerical integration using an earlier representation of $\mathcal{I}(x, y)$. This checks not only pointwise formulas but also support, endpoint behavior, and integration stability.

Level 5: literature and limiting checks. Compare special limits against known results. Possible checks include:

- recover the one-loop collinear kernel when one substructure is removed or replaced by a trivial hard factor,
- compare the one-loop module against the paper's formula for $I_c(x, m, M)$,
- compare suitable integrated counterterms against the diagonal-box results in the paper after choosing the matching hard function and mass convention,
- check small-mass limits against known logarithmic structures where available.

These are true external validations, because they compare your result against independently known information.

Level 6: direct loop-momentum validation. A direct numerical evaluation of the original differential integral would be the strongest check, but it is also the hardest. It would require a careful contour strategy or an equivalent Euclidean/Feynman-parameter formulation that preserves the delta constraints. This should not be the first numerical goal.

8.5 How the code should reflect this

The current scripts named `check_closed_vs_w.py` and `check_closed_vs_u.py` are useful, but their role should be renamed conceptually:

- they are internal regression checks,
- they are not the final validation of the result,
- they should be supplemented by checks against earlier representations and known limits.

A better future script layout would be:

Script	Purpose
<code>check_internal_representations.py</code>	compare closed, w , and u forms
<code>check_symbolic_identities.py</code>	verify algebraic identities with Symbolica
<code>check_residue_representation.py</code>	compare closed form to an earlier numerical representation
<code>integrate_test_hard_functions.py</code>	test convolutions with simple $G(x, y)$
<code>compare_literature_limits.py</code>	test known limits or known integrated results

8.6 Correct interpretation of the current tests

The current tests should be described as follows:

The tests verify that the implemented formulas for the closed form and the reduced parameter representations are mutually consistent. They are regression tests for the code and for algebraic transcription. They do not, by themselves, validate the closed form against the original two-loop integral.

This is the important correction.

9 Recommended next code additions

9.1 Make the tests stricter

After confirming the current formulas, replace the smoke tests by actual comparison tests:

```
def test_closed_vs_w_numerically():
    exact = closed_form(kin, eps)
    numeric = method1_w_integral(kin, eps)
    assert relative_error(numeric, exact) < mp.mpf("1e-25")
```

Do the same for `method2_u_integral`. If these fail, first inspect the relative errors printed by the scripts.

9.2 Add a symbolic expression module

Add `src/lsmethod/symbolic_expressions.py` with Symbolica versions of the central expressions:

- Δ_0 ,
- Δ_1 ,
- Δ_w ,
- $\Delta_x(w)$,
- C_u .

Then add tests that check identities such as

$$\Delta_1 = -[(1 + \kappa)\Delta_0 + \Delta_w]. \quad (24)$$

These are symbolic tests, not numerical tests.

9.3 Add a notebook only for exploration

A notebook is useful for interactive exploration, plots, and printing formulas. It should not be the only place where important code lives. If a calculation becomes important, move it into `src/lsmethod` or `tests`.

10 Daily workflow

A good working sequence is:

1. Update the LaTeX derivation in Overleaf.
2. Make sure every central equation has a stable label.
3. Update the matching function in `src/lsmethod`.
4. Update `docs/equation_map.yml` if a new equation is added.
5. Run `pytest`.
6. Run the relevant script in `scripts`.
7. Commit the change with Git.

Example:

```
pytest
python scripts/check_closed_vs_w.py
git add .
git commit -m "Check closed form against w representation"
```

11 Euler usage

The file `cluster/submit_scan.sh` is a starting point for SLURM submission. It should not contain derivations. It should only activate the environment and run a script.

```
sbatch cluster/submit_scan.sh
```

The script currently calls

```
python scripts/scan_random_points.py --npoints 1000 --eps 0.08
```

For Euler, you may need to adapt the environment activation depending on how your conda or mamba installation is exposed on the cluster.

12 Summary

The project has four layers:

Layer	Purpose
LaTeX labels	stable reference for equations
<code>src/lsmethod</code>	reusable implementation
<code>tests</code>	automatic correctness checks
<code>scripts</code>	manual runs, scans, and outputs

The main numerical idea is to build a validation ladder. The equality

$$\boxed{\mathcal{I}_{\text{closed}} = \mathcal{I}_w = \mathcal{I}_u} \tag{25}$$

is only the first internal regression check. The real validation is to compare the closed form to earlier representations of the original differential integral, to integrated test convolutions, and to known results or limits from the literature.

References and documentation

- Anastasiou and Sterman, *Removing infrared divergences from two-loop integrals*, arXiv:1812.03753.
- Pytest documentation: <https://docs.pytest.org/en/stable/>
- Python dataclasses documentation: <https://docs.python.org/3/library/dataclasses.html>
- Mpmath documentation: <https://mpmath.org/doc/current/>
- Python packaging guide for `pyproject.toml`: <https://packaging.python.org/en/latest/guides/writing-pyproject-toml/>
- Symbolica Python API documentation: https://symbolica.io/docs/guide/python_api.html